NEXTGEN QA

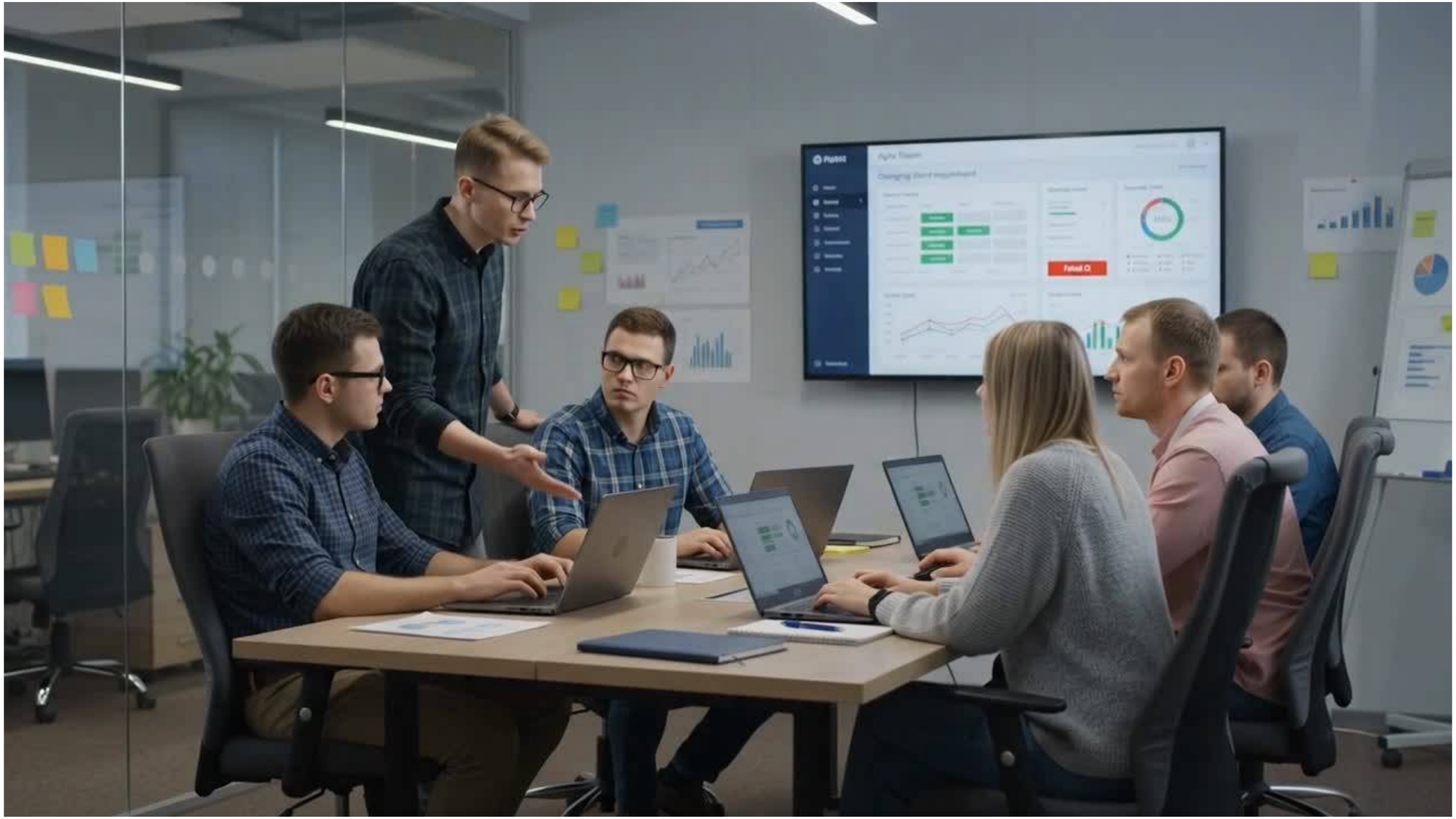
NextGenQA

From Agile conversations to executable, self-healing test suites

- **C1 · User Stories**
- **C2 · Test Generation**
- **C3 · Self-Healing**
- **C4 · RTM & Quality**

the closed loop: generate → run → analyse → heal → trace

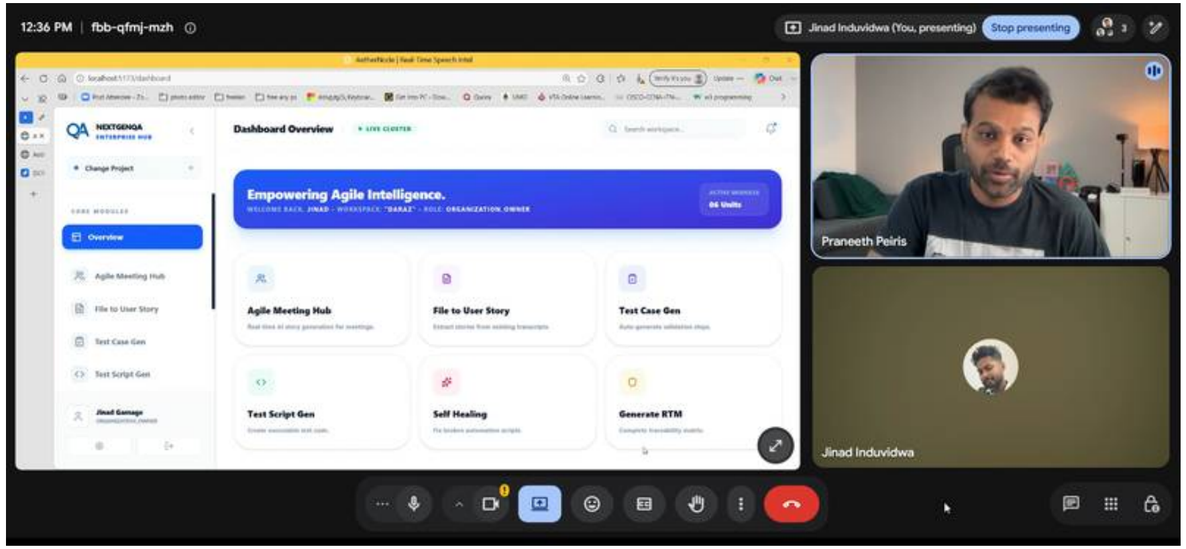




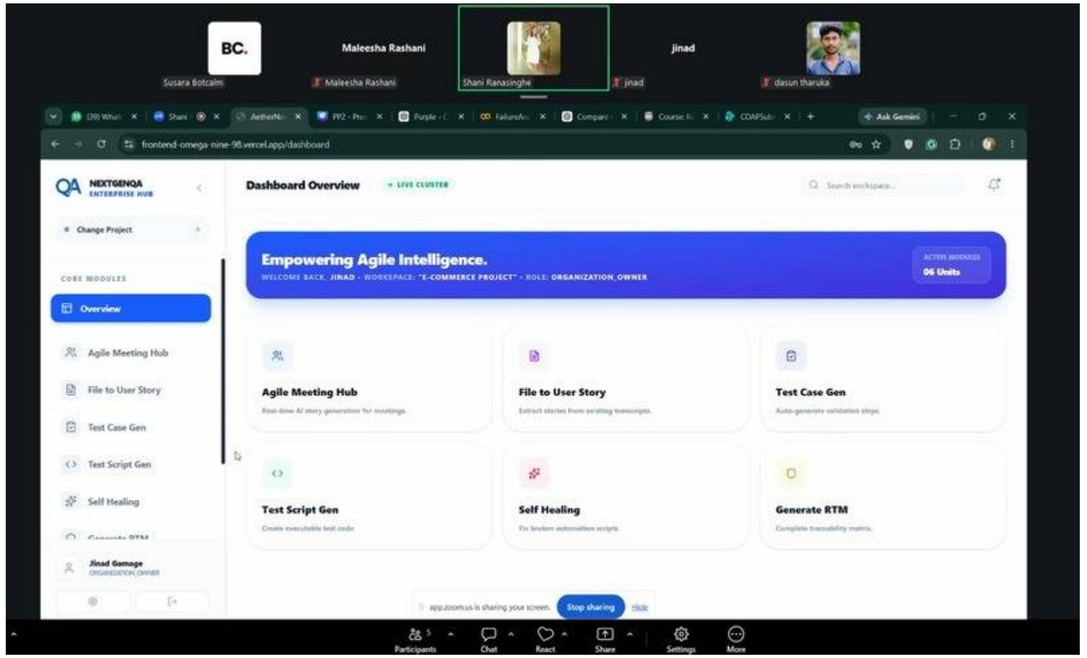
Comment Received on Progress presentation 1

Panel feedback after PP1 asked us to evaluate the actual output and bring evidence. We ran review sessions with practising QA engineers at two companies and paired their verdicts with measured results from the live pipeline.

How We Addressed It



Praneeth Peiris – Software Engineering Lead at trivago



Susara - QA Lead at Botcalm

Testing is the bottleneck of modern delivery

A fragmented, manual QA lifecycle

Requirements are captured in meetings, test cases are hand-written, code is hand-coded, failures are manually triaged — and traceability is a spreadsheet.

Every hand-off loses time & information

Meeting → story → test case → code → triage: each step is a manual translation while teams ship faster with Agile and CI/CD.

Automated suites decay as the UI changes

Flaky tests and brittle, hard-coded locators make automation expensive to keep alive.

30-40%

of total software project effort goes to testing & QA (industry-reported)

THE MEASURABLE PAIN WE TARGET

- Time from a spoken requirement to a passing, traceable automated test
- Cost of maintaining tests every time the application changes

Stakeholders QA engineers (primary users) · Developers · Product owners & Business Analysts · Agile teams & QA managers

SLOT 2 · RESEARCH GAP

Existing tools each solve one slice — not the loop

Test management

Jira · Zephyr · TestRail

Store test cases — but never generate them.

Record & playback

Selenium IDE · Katalon

Brittle scripts, hard-coded selectors, no requirement grounding.

LLM code generators

Zero-shot test writing

Plausible tests, hallucinated selectors — not grounded in the real DOM; no risk priority, no closed loop.

Traceability

Requirement → test mapping

Still a manually maintained spreadsheet — stale the moment anything changes.

THE GAP

“No integrated system takes a requirement from natural-language capture all the way to grounded, executable, self-healing tests with automatic requirement traceability.”

SLOT 2 · OBJECTIVES

Four components close the loop

C1

Intelligent User Story Generation

- Real-time meeting speech → transcript & speaker diarization
- Fine-tuned ModernBERT filters requirement utterances (9 classes)
- Conflict detection + BA validation → RAG-generated, INVEST-scored user stories

KEY NOVELTY

Realtime conflict and duplication detector

C2

Intelligent Test Case Generation

- Stories → Gherkin → executable Selenium / Playwright / Cypress code
- Grounded in the real DOM (crawler) + vision-guided Agent Explorer
- CI execution + ML risk model prioritises fragile flows

KEY NOVELTY

vision-guided agent exploration

C3

Failure Analysis & Self-Healing

- 9-class ML root-cause classifier for CI/CD failures
- Deterministic safety policy — prediction ≠ permission
- Approved application defects only → isolated draft-PR repair via GitHub MCP

KEY NOVELTY

self healing code base

C4

RTM & Quality Prediction

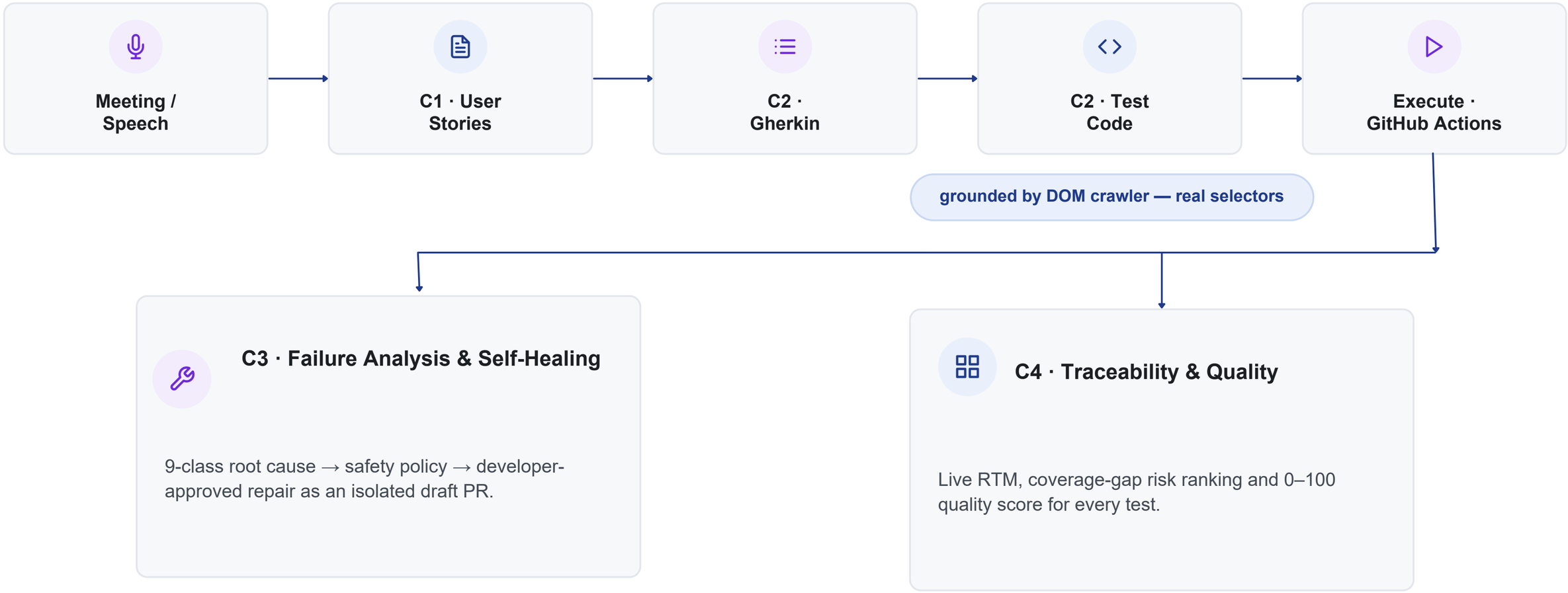
- Live Requirements Traceability Matrix — recomputed on every write
- ML quality score (0–100) for every test
- GitHub-grounded coverage agent runs the real test suite

KEY NOVELTY

GitHub-grounded live coverage agent — real numbers and Automated RTM

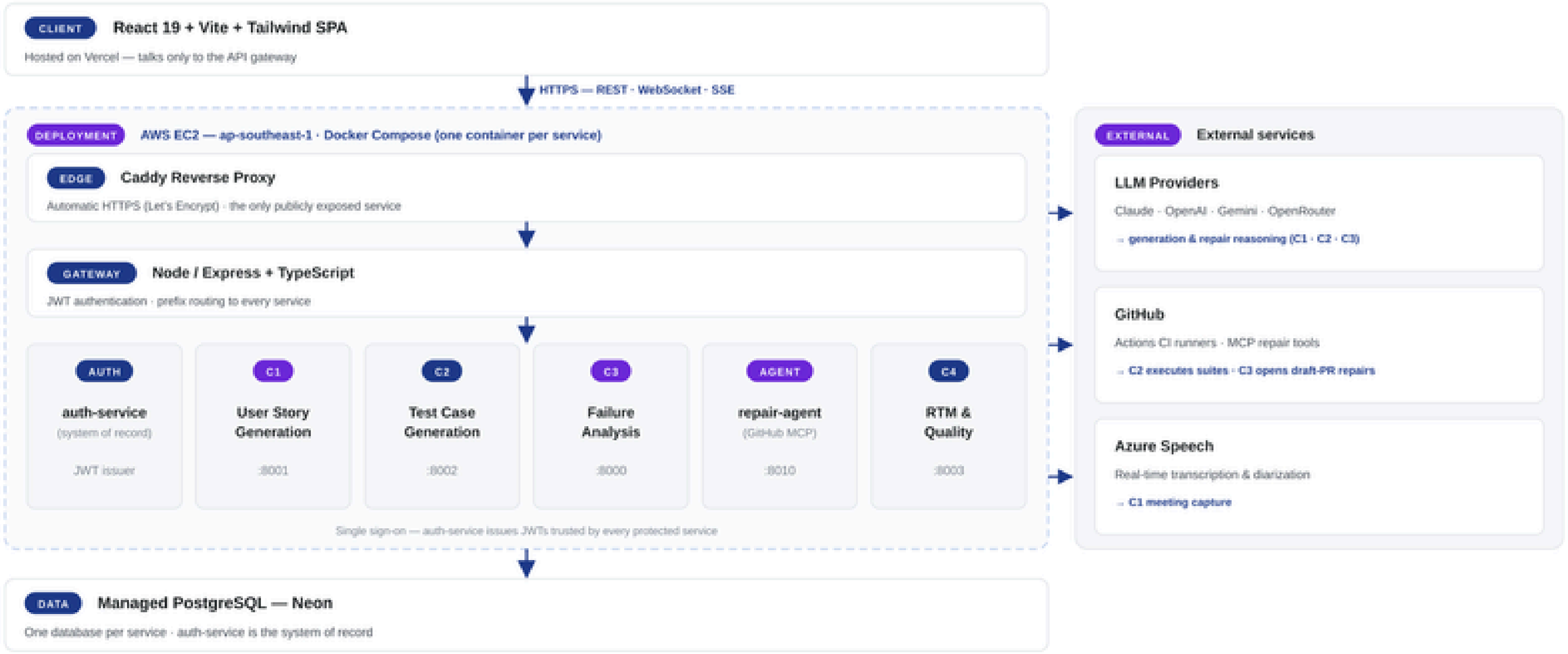
SLOT 3 · SOLUTION OVERVIEW

One pipeline: meeting → self-healing tests

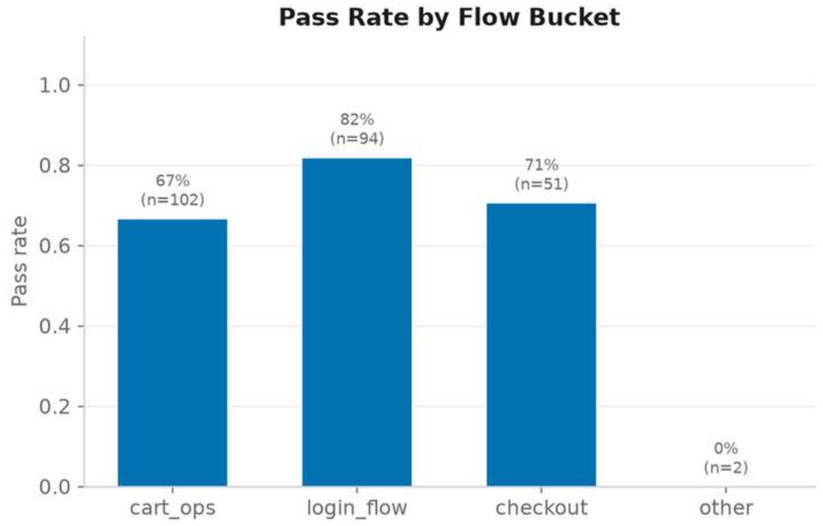


Run results feed back into risk prediction and self-healing — a closed loop, with a human approval gate at every stage.

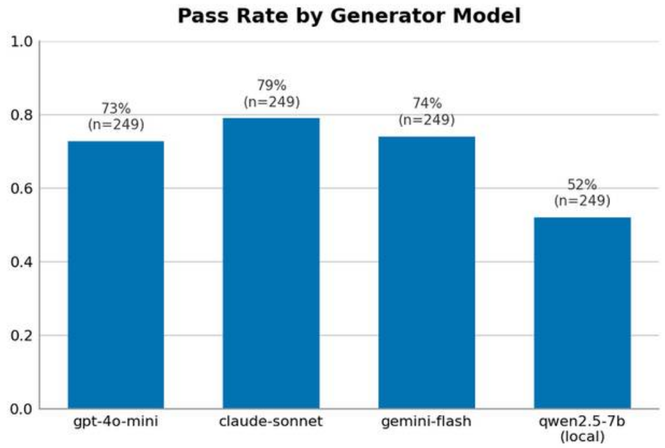
Cloud-deployed microservices, one secure edge

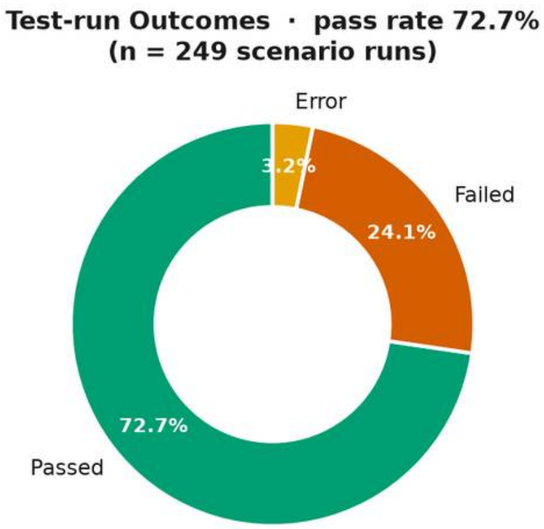


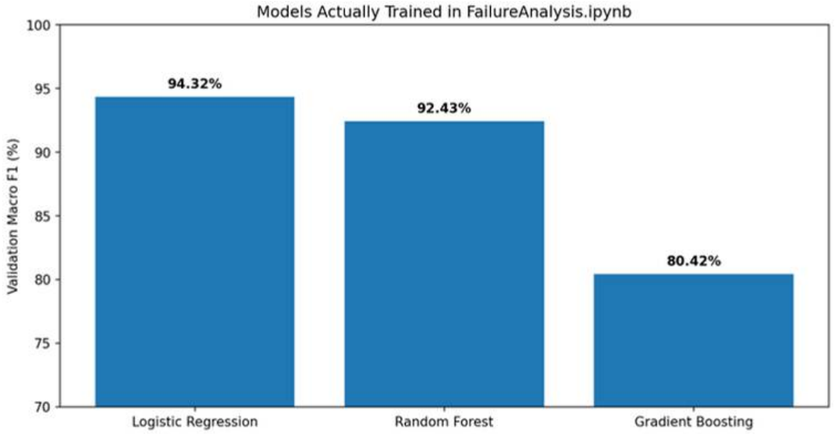
Measured from live executions - not projections

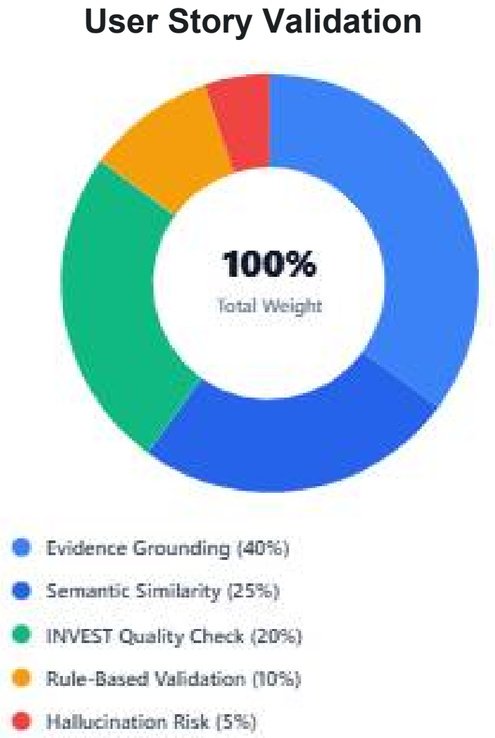


Test Pass rates









SLOT 5 · SAFETY & NFRs

Safe by design: prediction ≠ permission

C3 — SAFETY-GOVERNED SELF-HEALING

1. **ML classifier**
   9 root-cause classes predict why the pipeline failed
2. **Deterministic policy**
   only application_defect may proceed — 8 classes stay non-invasive
3. **Developer approval**
   mandatory human gate before any repository write
4. **Isolated publishing**
   auto-heal branch · one commit · draft PR — never main, never auto-merge

Headline safety metric — 0% unsafe GitHub modifications for all non-application failures

NON-FUNCTIONAL REQUIREMENTS

Security
JWT at the gateway · HTTPS everywhere · GitHub tokens encrypted at rest (Fernet) · least privilege. user role base authorization system

Reliability
Health probes + status page · deploy gate stops broken releases

Scalability & portability
Containerised, stateless services · managed Neon DB — moves to a bigger box without code change

Maintainability
Microservices with clear ownership · test-gated CI/CD · API-documented

Performance
Report generation & DOM crawl run off-request in worker threads

SLOT 6 · COMMERCIALIZATION

From research artifact to product

Target market

Software teams practising Agile + CI/CD — QA teams, dev shops, SaaS companies. Strong fit for regulated domains that must prove requirement → test traceability.

Value proposition

Compress requirement → passing, traceable test from hours to minutes — and cut test-maintenance cost via DOM-grounded generation + self-healing.

Business model

SaaS subscription per seat / project + usage-based LLM tier. On-prem option for enterprises. Land on QA-automation pain, expand across the QA lifecycle.

Differentiator

The closed loop point tools don’t offer — generate → run → analyse → heal → trace — plus LLM-agnostic generation (Claude · OpenAI · Gemini).

SLOT 6 · DEMO ROADMAP

What we will show you live

1. **Generate user story from meeting C1** — and generate Gherkin scenarios
2. **Generate Playwright test code** — grounded in crawled DOM selectors
3. **Execute the suite** — live results, screenshots, downloadable PDF report
4. **C3 classifies a failure** — indentify the root cause for failure
5. **C3 Self Healing** — and proposes a draft-PR repair
6. **C4 traceability matrix** — + live HTTPS status page & Swagger docs

NextGenQA

The closed loop from conversation to confidence.

generate → run → analyse → heal → trace

Project R26-SE-039 · SLIIT Faculty of Computing